

Esercizio 1: Sistema di controllo temperatura

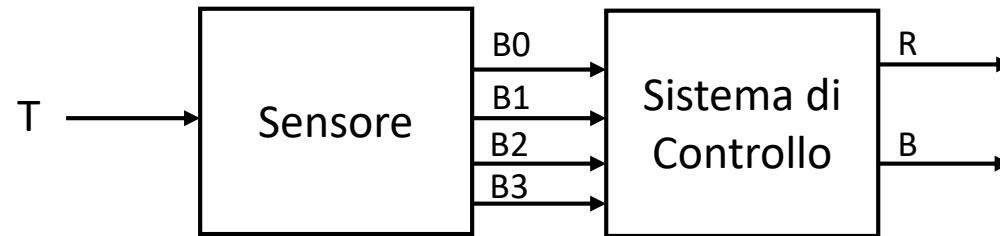
Un sensore fornisce 4 output: B0, B1, B2, B3. Ogni output e' associato ad un certo range di temperatura come riportato nella tabella sottostante. Progettare il sistema di controllo tale che:

- $T \geq 20^{\circ}\text{C} \rightarrow$ accendo il led rosso (R)
- $T < 10^{\circ}\text{C} \rightarrow$ accendo il led blu (B)

T [°C]	B3	B2	B1	B0	R	B
0	0	0	0	1	0	1
$0 \leq T < 10$	0	0	1	0	0	1
$10 \leq T < 20$	0	1	0	0	0	0
$T > 20$	1	0	0	0	1	0

Esercizio 1: Sistema di controllo temperatura

Per progettare la rete combinatoria bisogna valutare il numero di ingressi/uscite.



Ingressi: 4 $\rightarrow$ max $2^4 = 16$ combinazioni $\rightarrow$ Tabella di verita' lunga

Conviene utilizzare le mappe di Karnaugh per la sintesi logica sfruttando i "don't care", ovvero quelle combinazioni di ingresso che non influenzano in alcun modo l'uscita.

Uscite: 2 $\rightarrow$ 2 Mappe di Karnaugh

Esercizio 1: Sistema di controllo temperatura

T [°C]	B3	B2	B1	B0	R	B
0	0	0	0	1	0	1
$0 \leq T < 10$	0	0	1	0	0	1
$10 \leq T < 20$	0	1	0	0	0	0
$T > 20$	1	0	0	0	1	0

Compilo la mappa di Karnaugh inserendo “1” e “0” come da tabella di verita’.
Per esempio:

- L’unico “1” si ha per la combinazione B3 B2 B1 B0=“1000”
- Gli “0” si hanno per le combinazioni B3 B2 B1 B0=“0100”, “0010”, “0001”.
- Tutte le restanti combinazioni sono “don’t care”

Esercizio 1: Sistema di controllo temperatura

		LED ROSSO (R)			
B3 B2	B1 B0	00	01	11	10
	00	-	0	-	0
	01	0	-	-	-
	11	-	-	-	-
	10	1	-	-	-

 $R = B3$ sintesi "1"
 $R = B3$ sintesi "0"

		LED BLU (B)			
B3 B2	B1 B0	00	01	11	10
	00	-	1	-	1
	01	0	-	-	-
	11	-	-	-	-
	10	0	-	-	-

 $B = B3' \cdot B2'$ sintesi "1"
 $B = B0 + B1$ sintesi "1"
 $B = B3' \cdot B2'$ sintesi "0"

Esercizio 1: Sistema di controllo temperatura

Update specifica: se piu' di un $B_i=1$ accendi un led di errore e spegni sia il R e B. Se nessun $B_i=1$ accendi il led di errore e anche R e B.

$$\sum B_i > 1$$

R,B - OFF

E - ON

$$\sum B_i = 0$$

R,B - ON

E - ON

Esercizio 1: Sistema di controllo temperatura

B3	B2	B1	B0	R	B	E
0	0	0	0	1	1	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	0	1
0	1	0	0	0	0	0
0	1	0	1	0	0	1
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	0	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	0	1

 Dalla precedente tabella di verita' $\rightarrow$ E e' sempre "0"

 Compilata secondo l'update di specifica

$$\sum B_i = 0$$

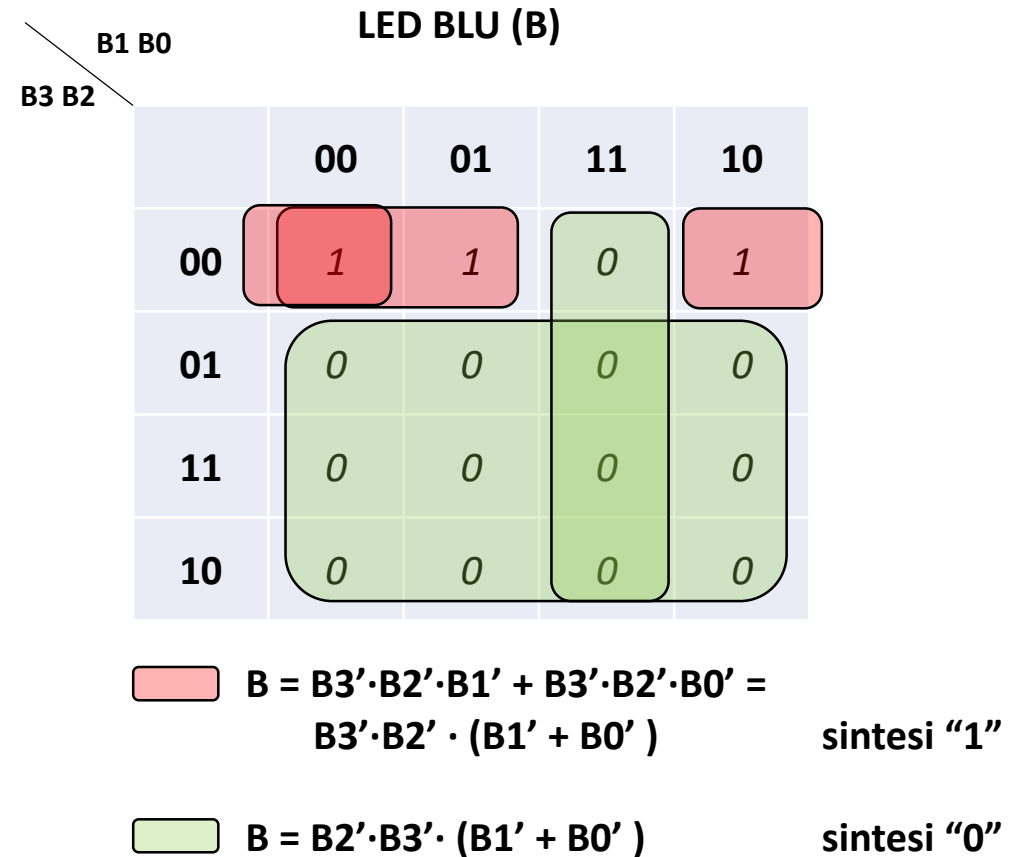
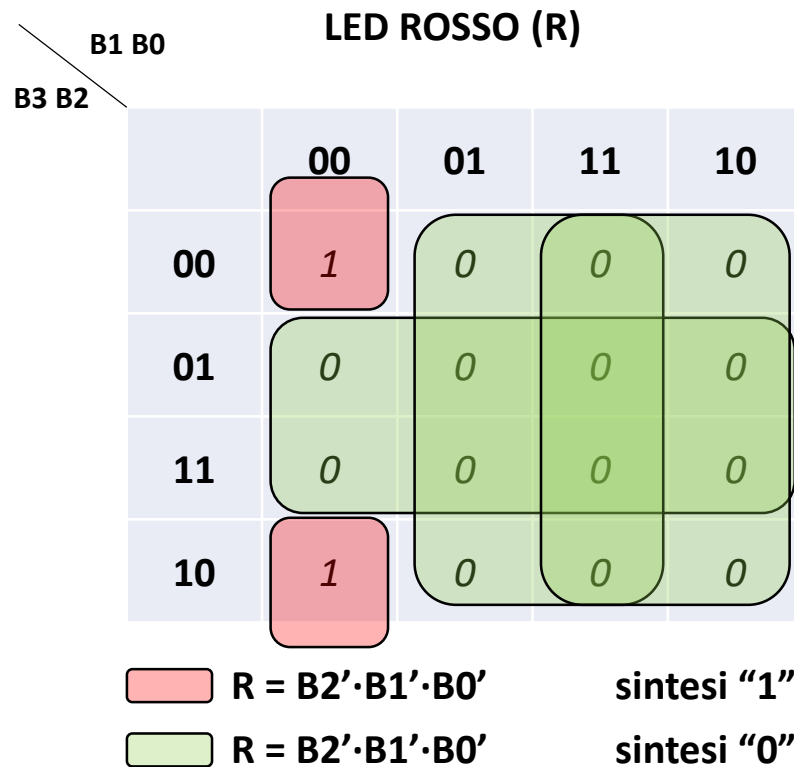
R,B - ON E - ON

$$\sum B_i > 1$$

R,B - OFF E - ON

Esercizio 1: Sistema di controllo temperatura

Risolvi le mappe di Karnaugh per le due uscite



Esercizio 1: Sistema di controllo temperatura

Risolviamo la mappa di Karnaugh per l'errore

		LED ERRORE (R)			
B3 B2	B1 B0	00	01	11	10
	00	1	0	1	0
01	0	1	1	1	1
11	1	1	1	1	1
10	0	1	1	1	1

 $E = (B3+B2+B1+B0) \cdot (B3+B2+B1'+B0) \cdot (B3+B2'+B1+B0) \cdot (B3'+B2+B1+B0)$ sintesi "0"

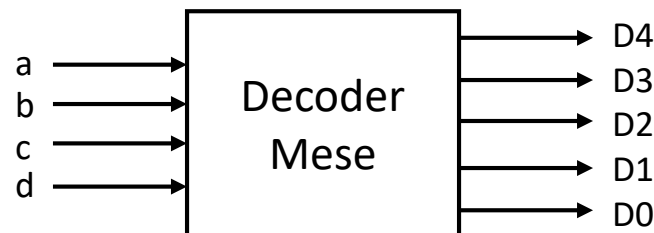
Esercizio 2: Decodificatore mesi dell'anno

Progettare una rete combinatoria che, a partire da un valore binario che codifica il numero del mese, calcoli il numero di giorni di tale mese.

Abbiamo 12 mesi, piu' il caso in cui febbraio e' bisestile. Per semplificare il problema consideriamo solamente 12 mesi. Eventualmente si puo' considerare febbraio bisestile come 13° mese.

Usiamo 4 bit in ingresso per codificare il numero del mese (valori ammessi da 1 fino a 13).

L'uscita varia tra solamente tra 28 e 31 $\rightarrow 2^5=32$, 5 bit sono necessari.



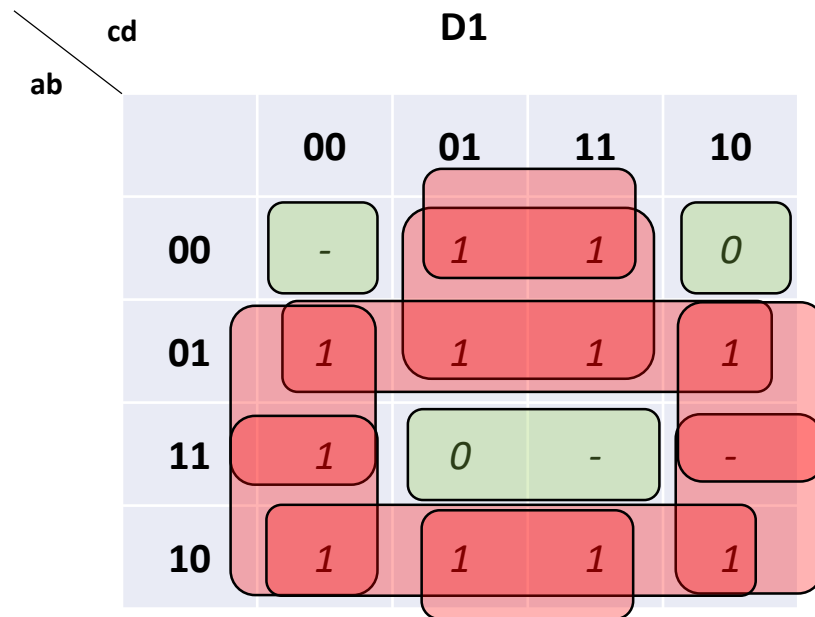
Esercizio 2: decodificatore mesi dell'anno

	Giorni	Ingressi				Uscite				
Mese		a	b	c	d	D4	D3	D2	D1	D0
Gennaio	31	0	0	0	1	1	1	1	1	1
Febbraio	28	0	0	1	0	1	1	1	0	0
Marzo	31	0	0	1	1	1	1	1	1	1
Aprile	30	0	1	0	0	1	1	1	1	0
Maggio	31	0	1	0	1	1	1	1	1	1
Giugno	30	0	1	1	0	1	1	1	1	0
Luglio	31	0	1	1	1	1	1	1	1	1
Agosto	31	1	0	0	0	1	1	1	1	1
Settembre	30	1	0	0	1	1	1	1	1	0
Ottobre	31	1	0	1	0	1	1	1	1	1
Novembre	30	1	0	1	1	1	1	1	1	0
Dicembre	31	1	1	0	0	1	1	1	1	1
Febbraio (Bisestile)	29	1	1	0	1	1	1	1	0	1

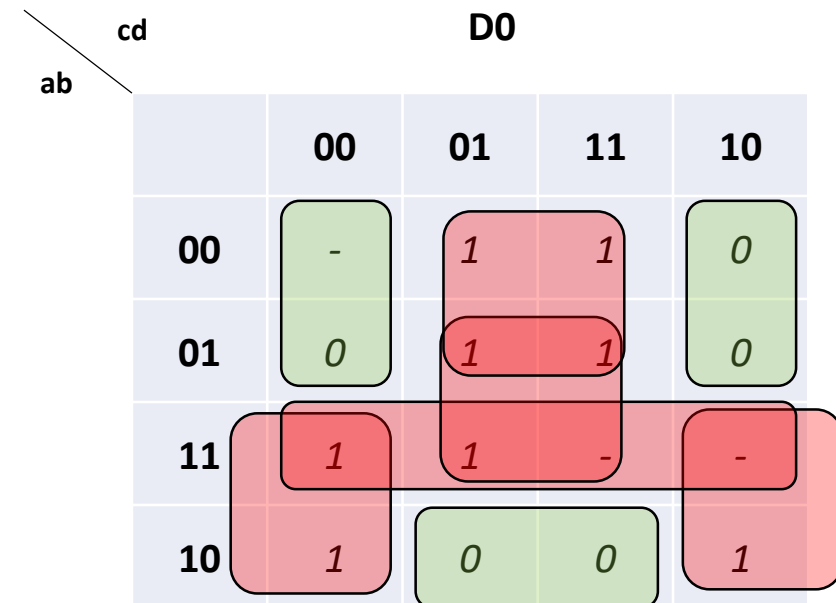
- I tre bit piu'significativi sono sempre a "1", quindi non c'e' bisogno di sintetizzarli.
- Rimangono solamente D1 e D0 sintetizzabili tramite mappe di Karnaugh.

Esercizio 2: decodificatore mesi dell'anno

Risolvi le mappe di Karnaugh per D1 e D0:



$D1 = ad' + b'd + a'b + bd' + ab' + a'd$
 $D1 = (a+b+d) \cdot (a'+b'+d')$
 $= \cancel{aa'} + ab' + ad' + a'b + \cancel{bb'} + bd' + a'd + b'd + \cancel{dd'}$



$D0 = ad' + ab + bd + a'd$
 $D0 = (a+d) \cdot (a'+b+d')$
 $= \cancel{aa'} + ab + ad' + a'd + bd + \cancel{dd'}$

Verifica: le espressioni della sintesi degli "1" coincidono con quelle ottenute per la sintesi degli "0"!!

Esercizio 3: comparatore gerarchico

Progettare una rete combinatoria che effettui la comparazione tra due stringhe binarie di lunghezza arbitraria. Il comparatore deve distinguere 3 diversi casi ($A > B$, $A = B$, $A < B$).

Svolgere l'esercizio seguendo un approccio simile a quello visto per il sommatore (half+full adder).

Per semplicità definire prima un circuito elementare ("half comparator") che effettui la comparazione tra due bit in modo tale da estendere poi il concetto ad un numero maggiore di bit. Tale circuito avrà in ingresso due bit, e deve essere in grado di:

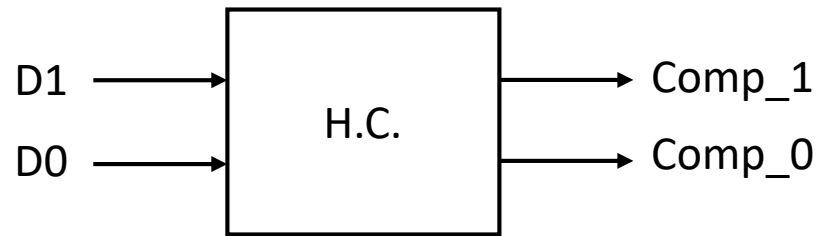
- Dire se i due bit sono identici
- Dire quale dei due è maggiore

Dovendo distinguere 3 diversi casi (bit uguali, primo bit maggiore, secondo bit maggiore), abbiamo bisogno di due bit di uscita.



Esercizio 3: comparatore gerarchico

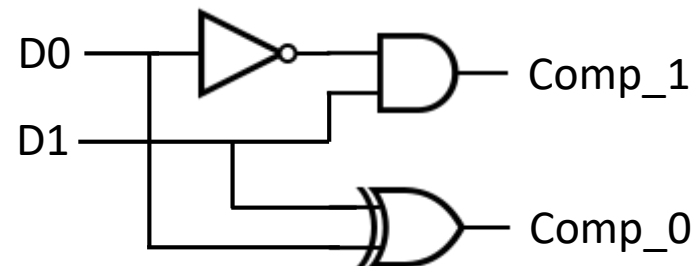
Definiamo quindi l'“half-comparator”.



Ingressi		Uscite		Risultato
D1	D0	Comp_1	Comp_0	
0	0	0	0	=
0	1	0	1	D0>D1
1	0	1	1	D0<D1
1	1	0	0	=

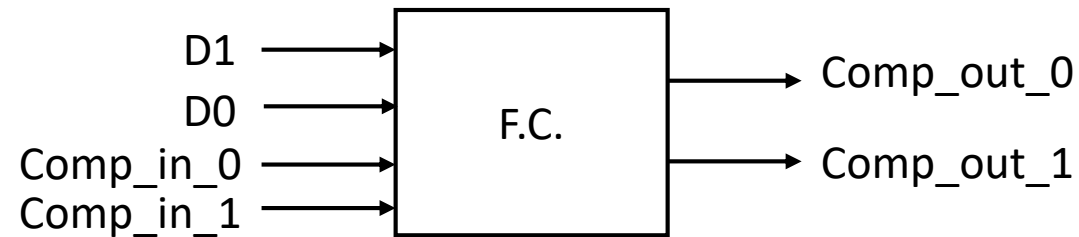
Data la semplicità, risolviamo direttamente tramite la tabella di verità:

- $\text{Comp_1} = D1 \cdot D0'$
- $\text{Comp_0} = D1 \oplus D0$



Esercizio 3: comparatore gerarchico

A questo punto, abbiamo un blocco elementare che compara due bit. Volendo estendere il concetto a piu' bit, bisognerà tenere in considerazione anche il risultato di comparazione del bit precedente partendo dal bit piu' significativo (simile a quanto visto nel sommatore). Estendiamo quindi il circuito per ottenere il "full-comparator". La tabella di verita' e' la seguente:



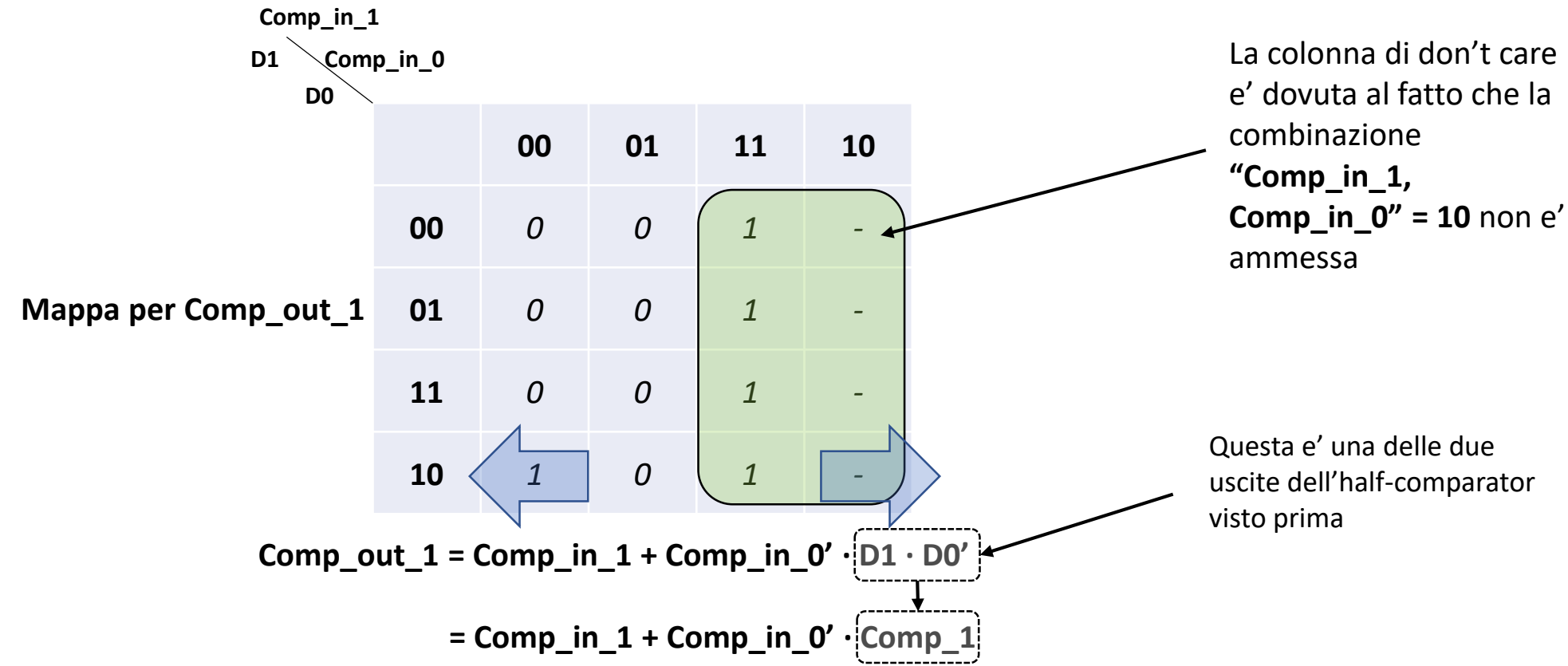
Ingressi				Uscite	
Comp_in_1	Comp_in_0	D1	D0	Comp_out_1	Comp_out_0
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	-	-	0	1
1	1	-	-	1	1

Le prime 4 righe sono uguali a quelle dell'half-comparator: infatti, i due bit "comp_in_1" e "comp_in_0" ci dicono che i bit precedenti sono uguali: il risultato non e' ancora stato deciso e puo' dipendere dai bit attuali

Le ultime due righe invece ci dicono che il risultato e' gia' stato deciso dai due bit precedenti: i bit attuali (D1 e D0) non hanno quindi piu' importanza.

Esercizio 3: comparatore gerarchico

Risolviamo le mappe di Karnaugh per le due uscite:



Esercizio 3: comparatore gerarchico

Risolviamo le mappe di Karnaugh per le due uscite:

Mappa per Comp_out_0

Comp_in_1 D1 \ D0 Comp_in_0					
		00	01	11	10
00	0	0	1	1	-
01	1	1	1	1	-
11	0	1	1	1	-
10	1	1	1	1	-

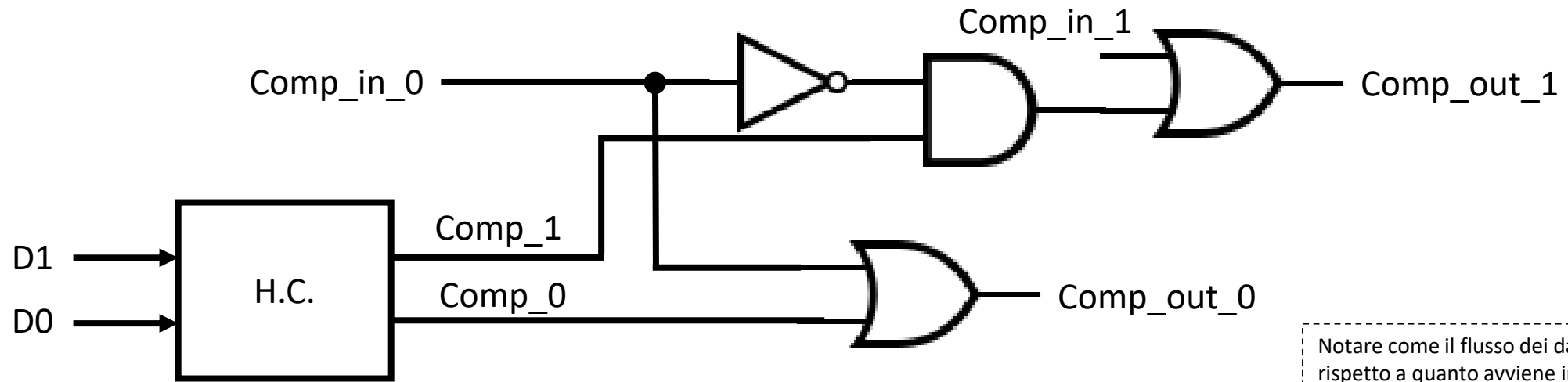
La colonna di don't care e' dovuta al fatto che la combinazione "**Comp_in_1**, **Comp_in_0**" = **10** non e' ammessa

Questa e' una delle due uscite dell'half-comparator visto prima

$$\begin{aligned}\text{Comp_out_0} &= \text{Comp_in_0} + \boxed{D1' \cdot D0 + D1 \cdot D0'} \\ &= \text{Comp_in_0} + \boxed{\text{Comp_0}}\end{aligned}$$

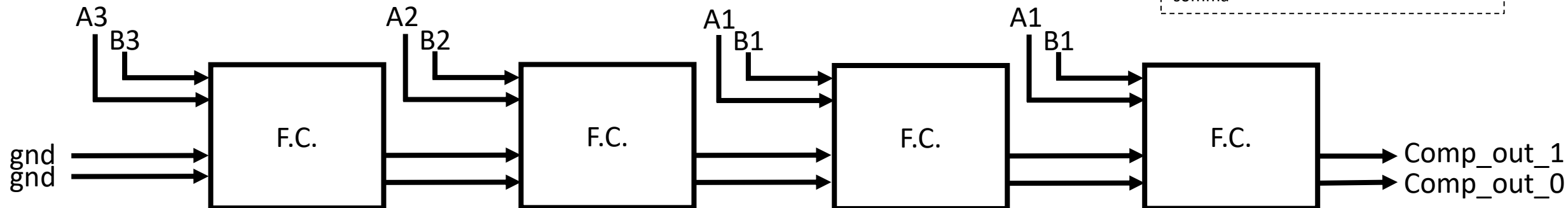
Esercizio 3: comparatore gerarchico

Lo schema circuitale del **full-comparator** e' il seguente:



Notare come il flusso dei dati sia opposto rispetto a quanto avviene in un sommatore: l'operazione di confronto ha inizio dal bit piu' significativo, al contrario di quanto avviene nell'operazione di somma

Esempio di comparatore binario a 4 bit:

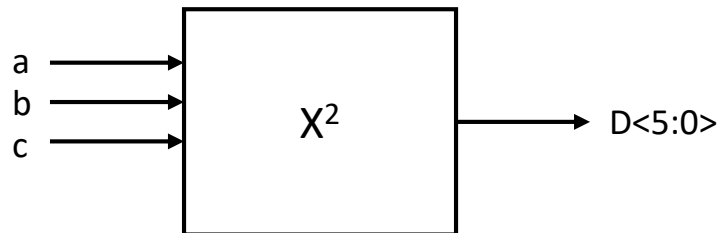


Esercizio 4: quadrato

Progetta un circuito che dato un numero in ingresso a 3 bit espresso in notazione complemento a 2, calcola il quadrato di tale numero in uscita.

Indispensabile comprendere il numero di bit in ingressi/uscite

- ingressi : 3bit
 - Spazio valori complemento a 2 e' definito da $[-2^{N-1}, 2^{N-1}-1]$ $\rightarrow$ range valori in ingresso = $[-4,3]$.
- Di conseguenza lo spazio d'uscita e' definito nell'intervallo $[0,16]$
 - Per esprimere 16 in complemento a 2: 1 bit (segno) + 5 bit (16 max)



Esercizio 4: quadrato

Ripasso:

Come convertire un numero intero in complemento a 2 ?

- se il numero e' positivo non cambia nulla
- Se il numero e' negativo
 - Trovo la rappresentazione in valore assoluto
 - Inverto i bit
 - Sommo 1

- Esempio -7 :

$$\begin{array}{r} 0000\ 0111 \rightarrow 1111\ 1000+ \\ 0000\ 0001= \\ \hline 1111\ 1001 \end{array}$$

-7 complemento a 2

Verifica? -7+7 ?

$$\begin{array}{r} 1111\ 1001+ \\ 0000\ 0111= \\ \hline 0000\ 0000 \end{array}$$

Esercizio 4: quadrato

a	b	c	Decimal in	Decimal out	D[5]	D[4]	D[3]	D[2]	D[1]	D[0]
0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	1
0	1	0	2	4	0	0	0	1	0	0
0	1	1	3	9	0	0	1	0	0	1
1	0	0	-4	16	0	1	0	0	0	1
1	0	1	-3	9	0	0	1	0	0	1
1	1	0	-2	4	0	0	0	1	0	0
1	1	1	-1	1	0	0	0	0	0	1

5 bit d'uscita → 5 espressioni logiche /mappe di Karnaugh

Possiamo analizzare meglio le colonne...

- $D[5]=D[1]=0$ Non serve sintetizzare (i quadrati sono sempre ≥ 0)
- $D[4]=1$ Solamente per la combinazione definita da $D[4]=ab'c'$
- $D[0]=c$ La sintesi circuitale e' un filo!

Esercizio 4: quadrato

Risolvi le mappe di Karnaugh per le due uscite:

		D[3]			
		00	01	11	10
a	bc				
0		0	0	1	0
1		0	1	0	0

- $D[3] = a'bc + ab'c$ sintesi "1"
- $D[3] = c(a+b)(a'+b')$ sintesi "0"
- $D[3] = c(a+b)(a'+b') = c(aa' + ab' + ba' + bb') = ab'c + a'bc$

Verifica: Coincide con l'espressione trovata per la sintesi degli "1"!!

		D[4]			
		00	01	11	10
a	bc				
0		0	0	0	1
1		0	0	0	1

- $D[4] = bc'$ sintesi "1"
- $D[4] = bc'$ sintesi "0"

Esercizio 5: sommatore BCD

Progettare un sommatore BCD a 2 cifre, utilizzando il sommatore binario.

La codifica BCD (Binary Coded Decimal) serve per rappresentare cifre decimali tramite il sistema binario. Ogni cifra decimale richiede quindi 4 bit (un *nibble*), in quanto bisogna codificare in binario numeri nel range $[0 \div 9]$.

La codifica BCD presenta numerosi vantaggi per quanto riguarda la visualizzazione dei dati per l'utente, alcuni di essi sono:

- Facile visualizzazione delle cifre tramite i famosi display “7 segmenti”
- Ha una corrispondenza diretta con il codice ASCII (standard per la codifica dei caratteri)



Esercizio 5: sommatore BCD

Tuttavia la codifica BCD ha lo svantaggio di essere meno efficiente rispetto al sistema binario “puro”. La perdita di efficienza cresce all’aumentare del range di valori visualizzato:

- Con 4 bit in BCD rappresento 10 numeri diversi, in binario ne rappresenterei 16.
- Con 8 bit in BCD rappresento 100 numeri diversi, in binario ne rappresenterei 256.
- Con 12 bit in BCD rappresento 1000 numeri diversi, in binario ne rappresenterei 4096.

L’efficienza dipende quindi dal numero di cifre BCD utilizzate:

- 1 cifra BCD: efficienza di $10/16 = 0.625$ (migliore)
- 2 cifre BCD: efficienza di $100/256 \approx 0.39$
- 3 cifre BCD: efficienza di $1000/4096 \approx 0.24$ (peggiore)

Esercizio 5: sommatore BCD

Vediamo ora come si esegue la somma con il sistema BCD confrontando i risultati di somma binaria e somma BCD.

	Operaz.	Primo operando						Secondo operando					Risultato				
		A3	A2	A1	A0			B3	B2	B1	B0		CO	S3	S2	S1	S0
Binario	4 + 3 = 7	0	1	0	0	+		0	0	1	1	=	0	0	1	1	1
BCD	4 + 3 = 7	0	1	0	0	+		0	0	1	1	=	0	0	1	1	1
Binario	4 + 9 = 13	0	1	0	0	+		1	0	0	1	=	0	1	1	0	1
BCD	4 + 9 = 13	0	1	0	0	+		1	0	0	1	=	1	0	0	1	1

Uguali

Diversi

13

Osserviamo che per risultati fino a 9, il risultato tra BCD e binario non cambia.

Viceversa, quando il risultato della somma restituisce un numero maggiore di 9, il risultato in BCD cambia in quanto “scatta” la decina.

Esercizio 5: sommatore BCD

Confrontiamo i codici:

- 13 in binario: 0000 1101
- 13 in BCD: 0001 (prima cifra) 0011 (seconda cifra)

Se leggo il valore 13 in BCD interpretandolo come una unica stringa binaria, il valore e' pari a 19.

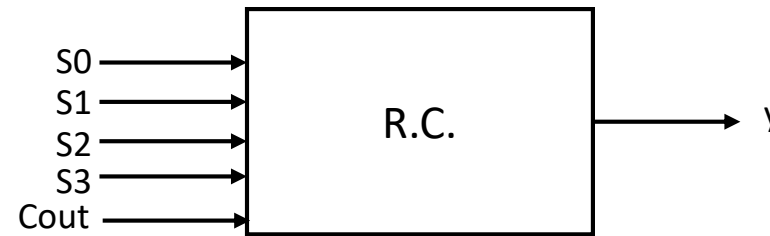
Abbiamo quindi una differenza pari a 6, rispetto al valore 13 espresso in codice binario puro: se sommassi 6 al valore 13 espresso in binario, ottengo una stringa a 8 bit che, letta in BCD, rappresenta il valore 13.

La stessa considerazione vale per tutti i numeri la cui somma e' maggiore di 9.

Di conseguenza, per progettare un sommatore BCD utilizzando sommatore binari, bisogna progettare una rete combinatoria che **riconosca un risultato maggiore di 9** e attivi una uscita per **sommare 6**.

Esercizio 5: sommatore BCD

Tale rete combinatoria avra' quindi 5 bit di ingresso (4 bit di somma piu' l'eventuale carry out), e un singolo bit di uscita (chiamiamolo "y").



Risolviamo tramite mappe di Karnaugh a variabili riportate: due mappe 4x4, una per la variabile riportata pari a 0 e l'altra per la variabile riportata pari a 1. Si compone poi il risultato finale tramite **espansione di Shannon**.

Come variabile riportata, scegliamo il carry out (Cout).

Esercizio 5: sommatore BCD

S3
S1 \ S2
S0

	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	0	0	1	1

$$Y0 = S3 \cdot S2 + S3 \cdot S1$$

Uscita y per Cout = 0
Gli '1' sono per tutte le
combinazioni
S3,S2,S1,S0 che
rappresentano un
numero maggiore di 9

S3
S1 \ S2
S0

	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$$Y1 = 1$$

Uscita y per Cout = 1
In tal caso ho sempre '1',
in quanto se Cout=1
automaticamente il
risultato e' maggiore di 9

Una volta ottenuti i risultati parziali, si compone il risultato finale applicando il teorema di espansione di Shannon:

$$\begin{aligned} y &= C'_{out} \cdot y_0 + C_{out} \cdot y_1 = \\ &= C'_{out} \cdot (S_3 S_2 + S_3 S_1) + C_{out} \end{aligned}$$

Esercizio 5: sommatore BCD

Schema circuitale

